Formación SQL 360°

Session 3 : Consultas Complejas

Facilitador: Flavio Bevilacqua

https://www.linkedin.com/flabevy

Temario de Clase:

- Subconsultas
- Joins
- Union vs Union All
- Case
- Funciones Windows
- CTEs

Subconsultas

SUBCONSULTAS

- El concepto Subconsulta hace referencia a una sentencia de Consulta contenida dentro de la sentencia principal.

- La subconsulta son aplicables a los comandos WHERE y HAVING.

- Dentro de una sola sentencia SQL se pueden especificar hasta 16 subconsultas y dentro de una subconsulta se pueden especificar subconsultas.

- En el proceso de ejecución del código, el motor de la base de datos resuelve:

SUBCONSULTAS AUTONOMAS

- En el proceso de ejecución del código, el motor de la base de datos resuelve primero la subconsulta para posteriormente ejecutar la sentencia principal.

- De modo tal que dicho proceso tiene ciertas similitudes con las funciones anidadas.

```
--lista los campos ProductID, ProductName,UnitsInStock,UnitPrice de todos
--los productos vendidos en una cantidad mayor e igual a 100
SELECT ProductID, ProductName,UnitsInStock,UnitPrice
FROM Products
WHERE ProductID IN
                    ( SELECT  ProductID
                      FROM [ORDER DETAILS]
                      WHERE Quantity>=100 ) ORDER BY ProductName
```

SUBCONSULTAS CORRELACIONADAS

- Funcionan fila por fila. Se ejecutan **muchas veces** porque dependen de un valor de la consulta principal para cada registro.
  - La lógica: "Para cada cliente, búscame su última compra específica".
  - Ejemplo Mostrar el nombre del empleado y, al lado, el total de ventas solo de ese empleado.
  - Ubicación común: En el SELECT.

Joins

JOINS

- El comando “Join” hace referencia al proceso de tomar datos de varias tablas e integrarlos al punto tal de poder generar un único resultado.

- Esto es posible dado a las relaciones y los campos en común que dichas tablas comparten, fundamento principal propio de una Base de Datos Relacional

```
SELECT * FROM Countries

SELECT * FROM States
```

100 %

Results | Messages

	CountryID	CountryName	CountryCode
1	2	United Kingdom	UK
2	3	United States	USA

	StateID	StateName	StateCode	CountryID
1	1	Texas	TX	3
2	2	Arizona	AZ	3

JOINS

- El comando “Join” acompañado por un comando “Select” nos permite generar información robusta de nuestro negocio

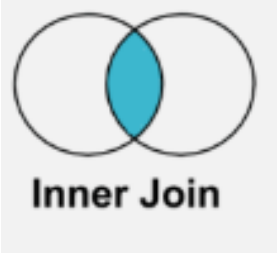
- Sintacticamente, debemos hacer referencia a los campos en común que comparten ambas tablas.

- La clausula FROM establece la tabla principal de nuestra consulta.

```sql
SELECT *
FROM HumanResources.Employee e
INNER JOIN Person.Contact c
       ON c.ContactID = e.ContactID
LEFT JOIN HumanResources.JobCandidate jc
       ON jc.EmployeeID = e.EmployeeID
INNER JOIN SALES.SalesPerson sp
       ON sp.SalesPersonID = e.EmployeeID
LEFT JOIN Sales.SalesOrderHeader soh
       ON soh.SalesPersonID = sp.SalesPersonID
LEFT JOIN Sales.SalesTerritory st
       ON st.TerritoryID = sp.TerritoryID
```

Tipos de Joins

INNER JOIN

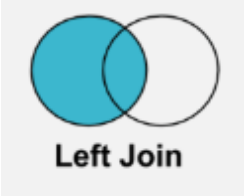


- El comando “INNER JOIN” también utilizado muchas veces como simplemente “JOIN”, busca coincidencias entre dos tablas, en función a una columna que tienen en común. De tal modo que solo la intersección se mostrara en los resultados.

```
SELECT *
FROM Empleados E
JOIN Departamentos D
ON E.DepartamentoId = D.Id
```

Nombre	DepartmentoId	Id	Nombre
Rafferty	31	31	Sales
Jones	33	33	Engineering
Heisenberg	33	33	Engineering
Robinson	34	34	Clerical
Smith	34	34	Clerical

LEFT JOIN

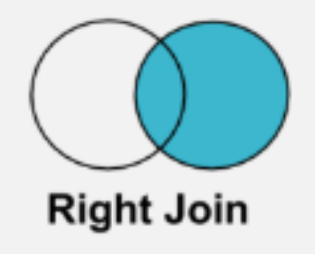


- El comando “LEFT JOIN” retorna todos los registro de la tabla izquierda junto a aquellos registros correspondientes a la tabla derecha, sobre los cuales encuentre una correlación
- Si no existe ninguna coincidencia para alguna de las filas de la tabla de la izquierda en la tabla derecha, de igual forma todos los resultados de la primera tabla de muestran.

```sql
SELECT
  E.Nombre as 'Empleado',
  D.Nombre as 'Departamento'
FROM Empleados E
LEFT JOIN Departamentos D
ON E.DepartamentoId = D.Id
```

Empleado	Departamento
Rafferty	Sales
Jones	Engineering
Heisenberg	Engineering
Robinson	Clerical
Smith	Clerical
Williams	NULL

RIGHT JOIN



- El comando “RIGHT JOIN” retorna todos los registro de la tabla derecha junto a aquellos registros correspondientes a la tabla izquierda, sobre los cuales encuentre una correlación.
- Si no existe ninguna coincidencia para alguna de las filas de la tabla de la derecha en la tabla izquierda, de igual forma todos los resultados de la primera tabla de muestran.

```sql
SELECT
  E.Nombre as 'Empleado',
  D.Nombre as 'Departamento'
FROM Empleados E
RIGHT JOIN Departamentos D
ON E.DepartamentoId = D.Id
```

Empleado	Departamento
Rafferty	Sales
Jones	Engineering
Heisenberg	Engineering
Robinson	Clerical
Smith	Clerical
NULL	Marketing

FULL JOIN

- El comando “FULL JOIN” se encarga de mostrar todas las filas de ambas tablas, sin importar que no existan coincidencias para sus respectivas filas.
- En caso que una fila de una de las tablas implicadas no tenga correlación con ningún otro registro de alguna otra tabla, asignara para cada valor faltante un NULL

```sql
SELECT
  E.Nombre as 'Empleado',
  D.Nombre as 'Departamento'
FROM Empleados E
FULL JOIN Departamentos D
ON E.DepartamentoId = D.Id
```

Empleado	Departamento
Rafferty	Sales
Jones	Engineering
Heisenberg	Engineering
Robinson	Clerical
Smith	Clerical
Williams	NULL
NULL	Marketing

Unions

UNION

- El comando “UNION” es una instrucción de conjunto que permite combinar resultados de instrucciones SELECT en un solo conjunto de resultados, anexando el resultado de las distantas Queries.
- Los requisitos a considerar en este punto son:
  - El numero de Orden de las columnas deben ser iguales en ambas consultas.
  - Los tipos de datos de las columnas correspondientes deben ser iguales o compatibles.

UNION vs ALL UNION

- Por default, el operador “UNION” elimina todas las filas duplicadas de los conjuntos de resultados. Sin embargo, si desea conservar las filas duplicadas, debe especificar la palabra clave “ALL” explícitamente.

- SQL SERVER, evalúa la duplicidad de filas en base al contenido de todo el registro en su conjunto.

CASE & END

CASE

➢ Las expresiones CASE se admiten en los módulos T-SQL compilados de forma nativa. En el ejemplo siguiente se muestra una forma de usar la expresión CASE en una consulta.

➢ En los ejemplos de código se usa una variable de tabla para crear un conjunto de resultados único. Esto solo es adecuado al procesar un número limitado de filas, ya que implica la creación de una copia adicional de las filas de datos.

Copiar

```
-- Query using a CASE expression in a natively compiled stored procedure.
CREATE PROCEDURE dbo.usp_SOHOnlineOrderResult
   WITH NATIVE_COMPILATION, SCHEMABINDING, EXECUTE AS OWNER
   AS BEGIN ATOMIC WITH  (TRANSACTION ISOLATION LEVEL = SNAPSHOT, LANGUAGE=N
   SELECT
      SalesOrderID,
      CASE (OnlineOrderFlag)
      WHEN 1 THEN N'Order placed online by customer'
      ELSE N'Order placed by sales person'
      END
   FROM Sales.SalesOrderHeader_inmem
END
GO

EXEC dbo.usp_SOHOnlineOrderResult
GO
```

Funciones Ventana

FUNCIONES VENTANA

- A diferencia del **GROUP BY** (que resume y agrupa filas), las Windows Functions permiten realizar cálculos avanzados manteniendo **todas las filas visibles**. Es como crear una métrica con "memoria" sobre lo que sucede en el resto de la tabla.
- ¿Qué las hace únicas?: Permiten que una métrica tenga su propio contexto (una "ventana" de datos) sin afectar la estructura general de la consulta.
- El motor **OVER:** Es la cláusula que define esa ventana.
  - **PARTITION BY:** Define el grupo (ej. "calcula esto por cada Ciudad"). Es nuestro "nuevo" agrupador que no colapsa filas.
  - **ORDER BY**: Establece el orden dentro de ese grupo, fundamental para que el cálculo tenga sentido lógico (ej. "el ranking del más vendido al menos vendido").

FUNCIONES CLAVES PARA NEGOCIO:

- **RANK():**
  - Asigna una posición (1º, 2º, 3º) según una métrica. Ideal para "Top N" de productos.

- **LAG() / LEAD()**:
  - Permite "mirar" la fila anterior o siguiente. Perfecto para calcular variaciones vs. el mes anterior sin procesos complejos..

- **SUM(...) OVER(...)**:
  - Acumulados históricos (Running Totals) fila por fila.

CTEs

CTEs (Common Table Expressions)

- Un CTE es una tabla temporal con nombre que existe solo durante la ejecución de tu consulta. Imagina que es como guardar el resultado de un paso intermedio en una pizarra para consultarlo inmediatamente después.
  - **SINTAXIS:** Se define con la palabra WITH, debe llevar nombre y finaliza con el AS

- **MODULARIDAD**: los CTEs son modulares, esto quiere decir que puedes definir un CTE para "Ventas" y otro para "Metas", y luego unirlos fácilmente al final.

- **MANTENIMIENTO:** Si la lógica del cálculo de "Margen" cambia, solo lo corriges en el CTE inicial, no en cinco lugares distintos del código.

Live Coding!

Gracias